

Lab of Algorithm

Assignment 9

From Greedy and Dynamic Programming to Maximum Flow: Resource Matching

Student ID: 411321344

Name: Wilkend AUGUSTE

Course: Lab of Algorithm

Date: June 16, 2026



National Dong Hwa University

Department of Computer Science and Information Engineering

1. Problem Background and Objectives

The assignment studies a resource matching problem in the form of student-task assignment. Given several students, several tasks, the list of tasks each student can accept, and capacity limits on tasks and also optionally students, the goal is to find the maximum number of valid assignments and output one valid matching.

Because the problem involves multiple sources, multiple targets, and capacity limits, it cannot be solved by a single-capacity knapsack or a simple greedy ordering. We therefore model it as a maximum flow problem and solve it with the Edmonds-Karp algorithm, while reviewing where Greedy and Dynamic Programming are appropriate.

2. Comparison of Greedy, DP, and Flow (Task 1)

Algorithm	Suitable Problem Structure	Relation to This Assignment
Greedy	Local optimal choices lead to a global optimum (the greedy-choice property and optimal substructure), e.g. unweighted activity selection.	Simple in-order matching may fail because an early choice consumes a scarce task needed by another student.
Dynamic Programming	Overlapping subproblems and optimal substructure where subproblem optima can be cached, e.g. weighted activity selection and 0/1 knapsack.	Matching has capacities on many nodes (source, students, tasks, sink), not a single knapsack constraint, so a one-dimensional DP table does not fit.
Maximum Flow	Multiple sources and targets with capacity-limited channels, where the goal is to maximize total throughput or the number of matches.	Models feasible student-task pairs as a flow network; the maximum flow value equals the maximum number of matches.

Answers to the five required sub-questions:

1. When does greedy work well? Greedy works when the problem has the greedy-choice property and optimal substructure - that is, choosing the locally best option at each step, without ever revisiting it, still yields a global optimum (for example unweighted activity selection sorted by finish time). Once a local choice can crowd out a better future choice, greedy no longer guarantees optimality.

2. Why is DP suitable for weighted activity selection or knapsack? These problems decompose into overlapping subproblems with optimal substructure: the take-or-skip decision for each item or activity reuses the optimal solution of a smaller subproblem. DP stores those subproblem optima to avoid recomputation and finds the best weighted combination in polynomial time, whereas greedy fails once weights are involved.

3. Why is Maximum Flow suitable for resource matching? Resource matching is essentially bipartite matching: students on the left, tasks on the right, feasible pairs as edges, each side bounded by a capacity. Modeled as a source $\rightarrow$ students $\rightarrow$ tasks $\rightarrow$ sink flow network, integer capacities guarantee an integer maximum flow (the integral flow theorem), the max flow value equals the maximum number of matches, and backward residual edges can even correct earlier wrong assignments.

4. Why does naive greedy not always give the maximum number of matches? Because greedy cannot backtrack. If a student grabs the only scarce task first, a later student who can accept only that task is left

unmatched, reducing the total. Maximum flow uses backward residual edges to push previously committed flow back and reassign it, effectively correcting matches, so it reaches the true optimum.

5. Why is this different from a simple knapsack problem? Knapsack has a single capacity (total weight) over independent items; resource matching is a many-to-many relationship where source $\rightarrow$ student, student $\rightarrow$ task, and task $\rightarrow$ sink each carry their own capacity, with flow conservation required at intermediate nodes. It is a multi-capacity problem on a graph rather than a one-dimensional capacity limit, so we use a flow network instead of knapsack DP.

3. A Case Where Greedy Matching Fails - Task 2

Greedy strategy: Process students in order; assign each student to the first acceptable task that still has capacity.

Test data: Student A accepts $\{T1, T2\}$, student B accepts only $\{T1\}$; tasks T1 and T2 each have capacity 1. (= greedy_fail_case.txt)

Greedy result: When processing A, T1 is the first acceptable task with capacity, so greedy assigns $A \rightarrow T1$. Then B can accept only T1, which is now full, so B cannot be matched. Greedy obtains only 1 match $\{A \rightarrow T1\}$.

Optimal result: Assigning $A \rightarrow T2$ and $B \rightarrow T1$ matches both students, giving 2 matches.

Why it fails: The early greedy choice $A \rightarrow T1$ consumes the scarce resource T1 (which only B can use, while A still had the alternative T2), leaving B with no option. Greedy has no backtracking and cannot move A to T2. Maximum flow can correct this through a backward residual edge.

Program verification: Running the program on greedy_fail_case.txt outputs Maximum Matching: 2, and its second augmenting path uses a backward edge through A (... $T1 \rightarrow A$...), which is exactly the correction of a previous match.

4. Flow Network Modeling

We model the problem as the directed graph below, with four layers: source, students, tasks, and sink.

source $\rightarrow$ student $_i \rightarrow$ task $_j \rightarrow$ sink

Edge Type	Capacity	Meaning
source $\rightarrow$ student	studentCapacity (1 in the basic version)	Limits how many tasks a student can be assigned.
student $\rightarrow$ task	1 (if the student accepts the task)	Represents one feasible (student, task) pairing.
task $\rightarrow$ sink	taskCapacity (1 in the basic version)	Limits how many students a task can accept.

Node indexing: node 0 = source; nodes 1..S = students; nodes S+1..S+T = tasks; the last node = sink. Except for source and sink, every intermediate vertex obeys flow conservation (inflow = outflow).

Why max flow = max matches: Each student $\rightarrow$ task edge has capacity 1, so each unit of flow represents one successful pairing. Flow conservation ensures no student sends more than its capacity and no task receives more than its capacity, so any feasible flow corresponds to a legal matching. By the integral flow theorem, integer capacities yield an integer maximum flow, which equals the maximum number of matches.

5. Edmonds-Karp Workflow and Pseudocode

Edmonds-Karp is an implementation of the Ford-Fulkerson method that always uses BFS to find the shortest (fewest-edge) augmenting path in the residual graph, augmenting repeatedly until no augmenting path remains.

```

maxFlow(source, sink):
    flow = 0
    while bfs(source, sink, parent):    # find shortest augmenting path
        pathFlow = +infinity
        v = sink                        # 1) find bottleneck
        while v != source:
            pathFlow = min(pathFlow, cap[parent[v]][v])
            v = parent[v]
        v = sink                        # 2) update residual graph
        while v != source:
            u = parent[v]
            cap[u][v] -= pathFlow        # forward edge decreases
            cap[v][u] += pathFlow        # backward edge increases
            v = parent[v]
        flow += pathFlow
    return flow

bfs(source, sink, parent):             # standard BFS on residual graph
    mark all parent = -1; parent[source] = source
    queue = [source]
    while queue not empty:
        u = queue.pop_front()
        for v in 0..N-1:
            if parent[v] == -1 and cap[u][v] > 0:
                parent[v] = u
                if v == sink: return true
                queue.push_back(v)
    return false

```

6. Manual Trace (Task 3)

Data: Students - A: {T1, T2}, B: {T1}, C: {T2, T3}, D: {T3}. Task capacities - T1 :1, T2 :1, T3 :1. All source -> student and task -> sink capacities are 1.

6.1 Vertices and Edges

Let s = source and t = sink. Edges (capacity) are listed below:

```

s -> A (1)  s -> B (1)  s -> C (1)  s -> D (1)
A -> T1 (1) A -> T2 (1)
B -> T1 (1)
C -> T2 (1) C -> T3 (1)
D -> T3 (1)
T1 -> t (1) T2 -> t (1) T3 -> t (1)

```

Capacity of each edge type: source -> student = 1, student -> task = 1, task -> sink = 1.

6.2 Augmenting Paths and Residual Updates

First augmenting path: BFS finds s -> A -> T1 -> t with bottleneck 1. After the update, the forward edges s->A, A->T1, T1->t each drop by 1 (to 0), and their reverse edges A->s, T1->A, t->T1 each rise by 1. Current flow = 1.

Second augmenting path: BFS then finds s -> C -> T2 -> t with bottleneck 1. Updating the corresponding forward and reverse edges raises the flow to 2.

(cont.) Third path: Next, $s \rightarrow D \rightarrow T3 \rightarrow t$ with bottleneck 1 raises the flow to 3. At this point B accepts only T1, but $T1 \rightarrow t$ is saturated and no $s \rightarrow t$ path can be found even through reverse edges, so BFS fails and the algorithm terminates.

Maximum flow and matching: Maximum flow = 3. One corresponding matching is $A \rightarrow T1$, $C \rightarrow T2$, $D \rightarrow T3$ (B is unmatched). This is exactly the program output for `basic_case.txt`.

7. Program Design - Tasks 4, 5, 6

7.1 Data Structures

We use an adjacency matrix `cap[N][N]` for residual capacities ($N = \text{number of students} + \text{number of tasks} + 2$). The parent `[]` array records the path found by BFS. A copy of the original capacity matrix is kept to recover the actual flow on each student $\rightarrow$ task edge and thus reconstruct the matching.

7.2 Main Functions

bool bfs (source, sink, parent): Performs BFS on the residual graph, finds one source $\rightarrow$ sink path, and records it in parent `[]`.

int maxFlow (source, sink): Repeatedly calls bfs, computes the bottleneck flow, updates the forward and backward edges, accumulates and returns the maximum flow, and also prints each augmenting path (bonus item).

7.3 Input and Output Format - Task 5

Input format:

```
<numStudents> <numTasks>
<name> <studentCapacity> <k> <task_1> ... <task_k> x numStudents
<taskName> <taskCapacity> x numTasks
```

Example input (`basic_case.txt`):

```
4 3
A 1 2 T1 T2
B 1 1 T1
C 1 2 T2 T3
D 1 1 T3
T1 1
T2 1
T3 1
```

Output: the program first lists each augmenting path, then prints the maximum matching number and one valid matching:

```
Maximum Matching: 3
Assignments:
A -> T1
C -> T2
D -> T3
```

7.4 Supporting Different Capacity Limits - Task 6

The program sets the source $\rightarrow$ student edge capacity to `studentCapacity` and the task $\rightarrow$ sink edge capacity to `taskCapacity`, so it naturally supports capacities greater than 1 without changing the core algorithm.

Which edges change and what they mean: The student $\rightarrow$ task edge stays at capacity 1 (one student can match a given task at most once). To express that a student can take several tasks, raise the source $\rightarrow$

student capacity (representing a workload or staffing limit). To express that a task needs several students, raise the task $\rightarrow$ sink capacity (representing task demand or resource quantity).

Example (multi_capacity_case.txt): A(cap1): {T1, T2}, B(cap2): {T1, T3}, C(cap1): {T2, T3}; T1:1, T2:2, T3:1. The program outputs a maximum matching of 4, namely A $\rightarrow$ T2, B $\rightarrow$ T1, B $\rightarrow$ T3, C $\rightarrow$ T2, with B taking two tasks and T2 receiving two students, all within capacity.

8. Test Cases, Results, and Verification

Test case	Expected	Actual	Purpose
basic_case.txt	3	3	Basic matching with all student/task capacities equal to 1.
greedy_fail_case.txt	2	2	Shows a naive greedy result (=1) is suboptimal.
multi_capacity_case.txt	4	4	Validates handling of student/task capacities greater than 1.
incomplete_matching_case.txt	1	1	Three students accept only T1 (capacity 1); not everyone can be matched.

Verification: Every test case's actual output matches the hand-derived expected value (see the output/directory). The basic case also agrees with the manual trace in Section 6, and the greedy-fail case uses a backward residual edge in its second augmenting path, demonstrating that the residual graph corrects matches.

9. Time and Space Complexity Analysis

Time complexity: Edmonds-Karp runs in $O(V \cdot E^2)$, where V is the number of vertices and E the number of edges. Each BFS is $O(E)$ ($O(V^2)$ with an adjacency matrix), and the number of augmentations is bounded by $O(V \cdot E)$.

V and E in this model: Let S be the number of students, T the number of tasks, and R the number of feasible pairings. Then $V = S + T + 2$ and $E = S$ (source $\rightarrow$ student) + R (student $\rightarrow$ task) + T (task $\rightarrow$ sink) = $S + T + R$. When each student can accept more tasks, R grows, E grows with it, and the running time can increase.

Space complexity: This implementation uses an adjacency matrix $\text{cap}[N][N]$, so space is $O(V^2)$; $\text{parent}[]$ is $O(V)$. Switching to an adjacency list would reduce space to $O(V + E)$, which is more economical for sparse graphs.

Bipartite special case: For unit-capacity bipartite matching, the maximum flow is at most $\min(S, T)$, so the number of augmentations is small, and the algorithm is fast in practice.

10. Analysis and Discussion - Task 7

1. Why is the maximum flow equal to the maximum number of matches? Each student $\rightarrow$ task edge has capacity 1, and flow conservation limits the flow each student sends, and each task receives to within its capacity. Every unit of source-to-sink flow must pass through one student $\rightarrow$ task edge, corresponding to one legal match; integer capacities guarantee an integer maximum flow, so the max flow value is exactly the maximum matching.

2. What do backward edges in the residual graph represent, and why do they allow adjustment? A backward edge represents existing flow that can be pushed back. When an augmenting path goes through a backward edge, it effectively cancels a previous match, frees that resource for a student who needs it

more, and reroutes the flow. This lets the algorithm correct greedy-style early mistakes and reach the global optimum.

3. Why can greedy matching fail? (linked to Task 2) As in Task 2, A grabs the scarce task T1 first, leaving B (who accepts only T1) unmatched; without backtracking, greedy gets just 1 match, while the optimum A->T2, B->T1 gives 2. Maximum flow corrects A's assignment through a backward edge and reaches the optimum.

4. Similarities and differences with the knapsack problem: Similarities: both are constrained optimization problems that can be described with take-or-skip decisions. Differences: knapsack has a single capacity (weight) over independent items and is typically solved with DP, while resource matching is a many-to-many problem on a graph with capacities on multiple nodes and flow conservation, solved with maximum flow.

5. If each student has a different preference score per task, is the current model sufficient? No. The current maximum flow model only maximizes the number of matches, ignoring quality or preference scores. To maximize total preference or minimize total cost, one needs Min-Cost Max-Flow, or the Hungarian algorithm for weighted bipartite matching.

6. What different problems do Maximum Flow and shortest path solve? Both operate on graphs, but shortest path finds a single minimum-weight route from a start to an end vertex, while maximum flow maximizes the total amount that can be sent from source to sink under all edge-capacity constraints - a whole-network throughput problem rather than a single-path one. Interestingly, Edmonds-Karp internally uses BFS (an unweighted shortest path) to choose augmenting paths.

11. Conclusion and Personal Observations

This assignment connected the optimization methods from across the semester: greedy is suited to problems where a local choice is already globally optimal, DP to weighted problems with overlapping subproblems and optimal substructure, and maximum flow to many-to-many matching and allocation problems with multiple capacity limits.

My main takeaway is that the value lies not in memorizing the Edmonds-Karp code, but in understanding how to convert a resource allocation problem into a flow network and how capacities describe real constraints. The backward residual edges made it concrete to me why the algorithm can fix greedy's mistakes, which is the core reason maximum flow outperforms a naive greedy approach.